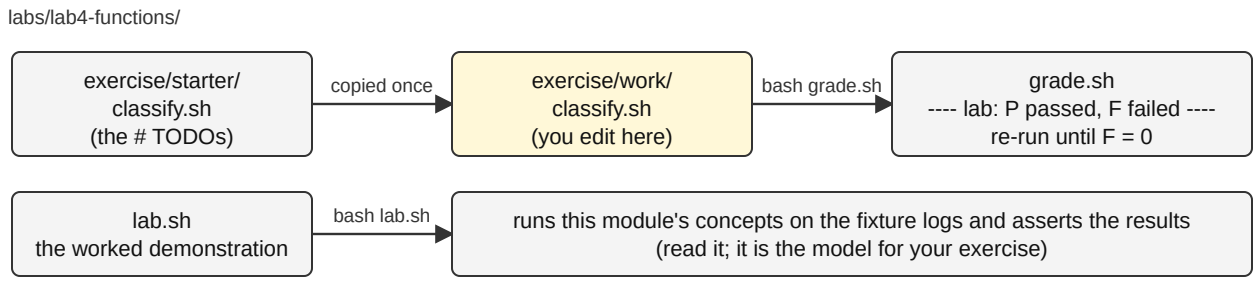
Lab 4 — Functions: A Predicate and a Producer (Instructor Version)

Instructor Note: Gray boxes and answers marked in red appear in the instructor copy only.

Topology



Environment Table

All paths are relative to `labs/lab4-functions/exercise/`.

File	What it is	You edit it?
`TASK.md`	The short statement of the exercise	No
`starter/classify.sh`	`is_valid_status` (always returns 0) and `status_class` (prints `FIXME`)	No — the grader copies it
`work/classify.sh`	**Your** copy (created by the first `grade.sh` run)	**Yes**
`grade.sh`	The grader: 9 checks. It sources your file	No
`work/REFLECTION.md`	Your two reflection answers	**Yes** (you create it)

Objectives

- **Part 1:** Prepare the exercise folder and read the grader's first report.
- **Part 2:** Build `is_valid_status` — a **predicate** that answers through its exit status and prints nothing.
- **Part 3:** Build `status_class` — a **producer** that answers on stdout and is captured with `$( )`.
- **Part 4:** Verify with the grader; write the two reflection answers.

Background / Scenario

A bash function can hand a result back in two ways, and `logsleuth`'s library uses both on purpose:

- A **predicate** returns an exit status — 0 for "yes", non-zero for "no" — so a caller can write `if is_valid_status "$c"` or `is_valid_status "$c" && …`. It prints nothing.
- A **producer** prints a value on stdout, so a caller can capture it with `cls=$(status_class "$c")`.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Mixing the two forces every caller to parse text. In this lab you write one of each: `is_valid_status CODE` is true for a three-digit code starting with 1–5; `status_class CODE` prints `4xx` for `404`, `5xx` for `503`, and `unknown` for anything that is not a valid code — and it *uses* the predicate to decide.

Required Resources

- A lab session or a shell where `bash tools-check.sh` reports `ready`.
- Module M04, step S04.3 (predicate vs producer) and S04.1–S04.2 (functions, `local`).
- A text editor. Documentation and AI assistance for lookups are allowed (say so in your reflection). Do **not** open `solution/`.

> **Instructor Note:** Budget 45 minutes; grading ≈ 5 minutes per learner. Observed: the starter scores `---- lab4: 2 passed, 7 failed ----` — the unconditional `return 0` accepts `200` and `404`, and everything else; the reference scores 9/9. Question 1 targets the most common misconception: a predicate that *prints* `true`/`false`.

Part 1: Prepare the exercise folder

Step 1: Run the grader once.

```
cd labs/lab4-functions/exercise
bash grade.sh
```

Expected: `---- lab4: 2 passed, 7 failed ----`.

Step 2: Open `work/classify.sh`.

Two functions. In `is_valid_status` the stub line `return 0` must go; in `status_class` the `printf 'FIXME\n'` must go.

Part 2: Build `is_valid_status`

Step 1: Let the test itself be the return value.

Replace the body of `is_valid_status` with a single test:

```
is_valid_status() {
  local code="$1"
  [[ "$code" =~ ^[1-5][0-9][0-9]$ ]]
}
```

How it works: a function returns the exit status of its **last command**. `[[ … =~ … ]]` exits 0 when the regular expression matches and 1 when it does not — so the test *is* the answer. Do not wrap it in `if … then return 0; else return 1; fi`; that adds nothing. The regex: first character 1–5, then two digits, nothing else.

Step 2: Try it by hand.

```
source work/classify.sh
is_valid_status 200; echo "status=$?"
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

```
is_valid_status 999; echo "status=$?"
is_valid_status 4xx; echo "status=$?"
```

Expected: `status=0`, `status=1`, `status=1` — and **nothing else printed**.

Part 3: Build `status_class`

Step 1: Guard the producer with the predicate.

```
status_class() {
  local code="$1"
  if is_valid_status "$code"; then
    printf '%sxx\n' "${code:0:1}"
  else
    printf 'unknown\n'
  fi
}
```

How it works: `if is_valid_status "$code"` tests the predicate's exit status. `${code:0:1}` is the first character of the code (substring expansion: offset 0, length 1). The `printf` is *inside* the `if`, so `nope` reaches the `else` branch and prints `unknown` rather than `nxx`.

Step 2: Try it.

```
status_class 404; status_class 503; status_class 200; status_class nope
cls=$(status_class 404); echo "captured [$cls]"
```

Expected: `4xx`, `5xx`, `2xx`, `unknown`, then `captured [4xx]`.

Part 4: Verify and reflect

Step 1: Run the grader until it is green.

```
bash grade.sh
```

Expected: `---- lab4: 9 passed, 0 failed ----`. The nine checks: `200` and `404` are valid; `999`, `4xx` and `20` are **not** valid; `404 → 4xx`; `503 → 5xx`; `200 → 2xx`; `nope → unknown`.

Symptom	Cause	Fix
`'4xx' is not valid` fails	The stub `return 0` is still the last line	Delete it so the `[[ ]]` test is the last command
`junk -> unknown` prints `nxx`	The `printf` is not guarded by the `if`	Put the `printf` inside the `then` branch
A predicate check fails although the regex looks right	The function prints `true`/`false` instead of returning	Remove the `echo`; the test's status is the answer

Step 2: Reflection.

Create `work/REFLECTION.md` and answer in your own words (3–5 sentences each):

Question 1: `is_valid_status` prints nothing and `status_class` returns nothing meaningful. Explain the rule behind that split, and say what would break for a caller like `if is_valid_status "$c"` if the predicate *printed* `true`/`false` instead.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

> **Answer:** **Met:** predicates answer yes/no through the exit status so `if`/`&&`/`||` can consume them; producers put data on stdout so `$( )` can capture it; a predicate that printed `true`/`false` would make `if is_valid_status …` branch on the *status* (still 0 → always "then") while the words leak into stdout. **Strong:** notes that mixing both forces every caller to parse text, or mentions stderr as the third channel, for messages. **Not met:** "one returns and one prints" with no consequence for the caller.
>
> *Calibration.* Met: "`is_valid_status` is used inside `if`, which reads the exit status, so its whole job is to exit 0 or 1 and stay silent; `status_class` is used inside `$( )`, which reads stdout. If the predicate printed `true`, `if` would ignore the word and the word would end up in whatever captured the output." Not met: "A predicate returns and a producer prints because that is the convention."

Question 2: In S04.3, `return 300` produced status 44. What does that tell you about using `return` to hand back a count, and which channel should a function use for data? Quote what you observed.

> **Answer:** **Met:** a return status is a single byte (0–255), so 300 became 44 (300 − 256); `return` carries a verdict, not data; data goes to stdout and is captured with `$( )`; quotes the observed `44`. **Strong:** adds that `$( )` of a function that only `return`s captures nothing (`captured=[]`), or that `return` exists for callers testing status. **Not met:** "bash can't return big numbers" without the channel conclusion, or no observation.

Hand-in and grading

Hand in `work/classify.sh`, `work/REFLECTION.md` and your final grader output. **Pass** = `0 failed` **and** both reflection answers name the mechanism. Unlimited attempts.

> **Instructor Note:** **Grading procedure (≈5 min per learner).**
>
> 1. `GRADE_TARGET=/path/to/their/work bash labs/lab4-functions/exercise/grade.sh` → require `---- lab4: 9 passed, 0 failed ----`.
> 2. Read `REFLECTION.md` against the descriptors. Record `A-05 PASS` only with 9/9 and both answers at Met or better.
> 3. **Acceptable variations:** `if [[ … ]]; then return 0; else return 1; fi` passes — coach to the one-line form, the idiom they will read everywhere; `case "$code" in [1-5][0-9][0-9]) return 0;; *) return 1;; esac` passes and is fine. A `status_class` that echoes `${code:0:1}xx` without the guard fails `nope → unknown` (`nxx`). A predicate that `echo`es `true`/`false` passes the `check`/`denies` tests only if it *also* returns the right status — it usually does not, and Question 1 targets exactly this.
> 4. **Model feedback (Not yet):** "`'4xx' is not valid` failed: your predicate still ends with `return 0` — delete the stub line so the `[[ ]]` test is the last command." Remediation: S04.3.
> 5. **Equivalent reassessment:** same grader; Q1′ "Why does the reference declare `local code="$1"` instead of using `$1` directly?" (Met: readability and naming, and `local` keeps it from leaking if the function later assigns) and Q2′ "Give one situation where a producer should write to stderr." (Met: a diagnostic or progress message that must not end up in captured data).

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note: **Appendix — reference solution (`solution/classify.sh`, observed 9/9):**

```
#!/usr/bin/env bash
# Lab 4 solution — a predicate (status) and a producer (stdout).

is_valid_status() {
  local code="$1"
  [[ "$code" =~ ^[1-5][0-9][0-9]$ ]]
}

status_class() {
  local code="$1"
  if is_valid_status "$code"; then
    printf '%sxx\n' "${code:0:1}"
  else
    printf 'unknown\n'
  fi
}
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.